BONUSAUFGABE 1

IoT-Echtzeitanalyse mit RabbitMQ

Python, Pika und AMQP

Architektur, Fehlerbehandlung und Live-Demonstration

Gruppenmitglieder: vor dem Vortrag ergänzen

Architektur und Nachrichtenfluss

Getrennte Prozesse kommunizieren über RabbitMQ

QUELLE	VERMITTLUNG	VERARBEITUNG
Drei Sensortypen	Topic: sensors	Umgebung / Maschine
Analyse-Consumer	Direct: alarms	Alarm-Consumer
Verarbeitungsereignisse	Fanout: telemetry	Monitoring-Consumer
Ablehnung / Ablauf	Direct: dlx	Dead Letter Queue

Routing und Exchange-Wahl

Der Broker entscheidet, welche Analyse einen Messwert erhält

```
sensors.temperature.temp-01
Binding Umgebung: sensors.temperature.*
                  sensors.humidity.*
Binding Maschine: sensors.vibration.*
```

Topic erlaubt fachliche Muster. Direct wählt exakte Alarmstufen. Fanout verteilt alle Monitoringereignisse.

Alternative: Fanout für Sensoren würde unnötige Kopien in beide Analysequeues senden.

JSON-Vertrag und Anomalien

Pflichtfelder stehen direkt in der Sensornachricht

```
{
  "sensor_id": "temp-01",
  "timestamp": "2026-10-21T10:00:00Z",
  "type": "temperature",
  "value": 92.5
}
```

Temperatur
über 80 °C: WARNING
über 100 °C: CRITICAL

Auch Feuchtigkeit und Vibration besitzen zwei Alarmstufen.

Grenzwerte sind Annahmen für die Simulation.

Senden und Bestätigen

Brokerannahme und erfolgreiche Verarbeitung sind getrennte Schritte

```
channel.confirm_delivery()
properties = pika.BasicProperties(
    delivery_mode=2)
channel.basic_publish(
    ..., properties=properties, mandatory=True)
```

Durable Topologie und persistente Nachrichten sichern den Brokerpfad. Der Consumer bestätigt seine Verarbeitung anschließend manuell.

Nach dem ersten Versuch höchstens fünf Retries: 1, 2, 4, 8 und 10 Sekunden.

Aggregation und Fehlerpfad

Temperaturwerte bleiben bis zum Fensterabschluss unbestätigt

01 Zehn Sekunden Empfangszeit bilden ein Fenster. Der Timer läuft auch ohne neue Nachricht.

02 Das bestätigte Aggregat enthält Anzahl, Mittelwert, Minimum und Maximum.

```
_telemetry(channel, topology, event)
for pending in window.pending:
    _telemetry(
        channel, topology,
        pending.event)
    channel.basic_ack(
        delivery_tag=pending.tag)
```

```
channel.basic_nack(delivery_tag=tag, requeue=False)
```

Dauerhafte Fehler nutzen nach begrenzter Fehlerbehandlung den DLX. Code aus flush_window und _reject, gekürzt.

Priorität, TTL und Monitoring

MECHANISMUS	KONFIGURATION	WIRKUNG
Alarmpriorität	Queue: max 9 Nachricht: 3 / 9	CRITICAL vor wartenden WARNING
Frische der Daten	TTL: 60.000 ms	Abgelaufene Werte in die DLQ
Monitoring	Eigene Telemetriequeue	Erfolge, Typen, Alarme, Fehler und mittlere Zeit

Prefetch 1 begrenzt die Vorabauslieferung der Alarme. Laufende Verarbeitung wird nicht unterbrochen.

Live-Demonstration

Vier Minuten im vorbereiteten Terminal

```
python -m iot_analysis run
python -m iot_analysis demo --scenario invalid
python -m iot_analysis demo --scenario crash
python -m iot_analysis demo --scenario priority
```

01 Normalbetrieb und Zehn-Sekunden-Aggregat beobachten.

02 DLQ-Eintrag und erneute Zustellung nach Consumer-Ausfall zeigen.

Verhalten in den Fehlerszenarien

AUSLÖSER	BESTÄTIGUNG	NACHRICHTENWEG
Ungültige Nachricht	NACK, requeue=False	DLX und DLQ
Consumer endet vor ACK	Weder ACK noch NACK	Erneute Zustellung
Dauerhafter Prozessfehler	Nach 3 Versuchen NACK	DLX und DLQ

Ein Consumer-Absturz allein führt bei Classic-Queues zunächst zur Requeue unbestätigter Lieferungen.

Erkenntnisse und Grenzen

01 Broker-Routing trennt die Fachanalysen. Eigene Alarme und Telemetrie halten Dienste unabhängig.

02 ACKs, begrenzte Versuche und DLQ machen das Verhalten bei bekannten Fehlern nachvollziehbar.

03 Mehrfachzustellung bleibt möglich. Ein einzelner Broker und Auswertungen im RAM begrenzen die Lösung.

Nächste Schritte: persistente idempotente Verarbeitung und Brokerreplikation nach Bedarf.